DATABASE PROJECT REPORT

Gym Membership & Personal Training Planner
(Project ID: #9)

A. PROJECT IDENTITY

Team Name	Dong Vui
Project ID & Title	**#9: Gym Membership & Personal Training Planner**
Course	**INT1313 - Database Basics**
Instructor	**Lê Hà Thanh**
Academic Year	**2026-2027 (Semester 1)**

Full Name	Student ID	Class	Domain Responsibility

B. REPORT STRUCTURE

1. Phase 1: Conceptual SRS & EER Design (ISO/IEC/IEEE 29148 Standard)

1.1 System Objective & ISO/IEC/IEEE 29148 Requirements

The Gym Membership & Personal Training Planner is a relational database for managing core gym operations. It covers three areas: Member & Membership Billing, Trainer Scheduling & Booking, and Facility Maintenance. The database stores member, staff, schedule, and billing data, and enforces data integrity through PK/FK, CHECK, and UNIQUE constraints, with triggers for scheduling-conflict rules.

The requirements below are organized into Functional Requirements (FR) and Non-Functional Requirements (NFR), following the requirement structure of ISO/IEC/IEEE 29148:

- **Functional Requirements:**

  FR-MEM (Member & Billing Domain): Record member registrations, daily walk-in tickets, membership plans, time-bound membership contracts, and billing records linked to each visitor.

  FR-SCH (Trainer & Session Scheduling): Manage group class schedules, private 1-on-1 PT bookings, and room assignments, and reject room or trainer schedule conflicts using triggers.

  FR-FAC (Facility & Asset Maintenance): Track equipment, record repair logs for each machine (weak entity), and record which staff member signed off each log.

- **Non-Functional Requirements:**

  NFR-INT (Data Integrity): Enforce referential integrity across all 16 relations using PK/FK constraints, with ON DELETE RESTRICT on financial tables to prevent orphaned or lost billing records.

  NFR-PER (Query Performance): Reduce query time for frequent scheduling and check-in lookups by creating indexes on search and time-range columns (e.g., Room_ID, Trainer_ID, Start_Time).

  NFR-SEC (Data Security): Store member contact data (phone, email) with UNIQUE constraints to prevent duplicate accounts, and store profile photos as binary data inside the database.

  NFR-AVA (Recoverability): Support data recovery through regular database backups.

1.1.1 Core Entity Definitions & Business Semantics

- **Visitor**: The superclass entity representing any individual entering the gym facility. Identified by Visitor_ID (Primary Identifier). Stores shared demographic and contact attributes (First_Name, Last_Name, Phone).
- **Member**: A specialized subclass of Visitor representing registered gym members with long-term contracts. Inherits Visitor_ID (PK/FK) and adds member-specific attributes (Email, Birth_Date, Photo, Registration_Date).
- **Daily Pass Ticket**: A specialized subclass of Visitor representing single-visit daily pass customers. Inherits Visitor_ID (PK/FK) and adds pass-specific attributes (Visit_Date, Daily_Rate).
- **Membership Plan**: Defines the membership packages offered by the gym (e.g., Monthly, Annual, VIP). Attributes include Plan_ID (Primary Identifier), Plan_Name (UNIQUE), Duration_Months (CHECK > 0), Price (CHECK >= 0.00), and Description.
- **Membership**: A relationship contract linking a registered Member (Visitor_ID) to a Membership_Plan. Identified by Membership_ID.

- **Billing Log**: Records each payment. Linked to Visitor via Visitor_ID [FK, NOT NULL], so every payment is tied to an identified visitor; Membership_ID is filled only for membership payments.
- **Staff:** The superclass entity representing all gym employees. Identified by Staff_ID (Primary Identifier), tracking Name, Role ('Trainer', 'Receptionist', 'Manager'), and Monthly Salary (CHECK > 0).
- **Trainer / PT:** A specialized subclass of Staff representing Personal Trainers. Possesses unique attributes Specialty and Certification, responsible for leading group classes or coaching private 1-on-1 sessions.
- **Receptionist:** A specialized subclass of Staff responsible for front-desk shift management (Shift: Morning, Evening, Night) and signing off on hardware maintenance logs.
- **Room:** Manages physical training spaces. Identified by Room_ID (Primary Identifier), tracking Room_Name (UNIQUE), Capacity (CHECK > 0), and Floor_Location.
- **Class:** Represents public group classes (Zumba, Yoga, HIIT) scheduled by the gym. Attributes include Class_ID (Primary Identifier), Class_Name, Room_ID, Trainer_ID, Start_Time, and End_Time.
- **Class Registration**: An associative entity representing member enrollments in group classes (linking Member via Visitor_ID).
- **PT Booking**: Manages private 1-on-1 coaching sessions between a Member (Visitor_ID) and a Trainer in a specific Room.
- **Equipment:** Manages workout machines. Identified by Equipment_ID (Primary Identifier), tracking Equipment_Name, Room_ID, Purchase_Date, and Operational_Status ('Active', 'Under Maintenance', 'Broken').
- **Maintenance Log:** A weak entity representing service and repair history on a specific machine. Identified by composite key {Equipment_ID, Log_no} (where Log_no is a partial key). Tracks Maintenance_Date, Cost (CHECK >= 0), Description, and Logged_By (receptionist/manager FK).
- **Session Equipment Usage:** An associative relationship entity representing the allocation of specific machines to group classes or private PT bookings. Identified by Usage_ID under an Exclusive Arc XOR constraint between Class_ID and Booking_ID.

1.1.2 Explanation of Technical Terms

- **Entity & Attribute:** An entity represents an independent physical or abstract real-world object (e.g., MEMBER, EQUIPMENT). An attribute represents a descriptive property of that entity (e.g., Phone and Email).
- **Enhanced Entity-Relationship (EER):** An extension of traditional ER modeling incorporating object-oriented concepts such as attribute inheritance, disjoint specialization (d), and weak entity dependencies.
- **Disjoint Specialization (d):** A constraint stipulating that an occurrence of superclass STAFF can belong to at most one subclass (TRAINER or RECEPTIONIST).
- **Total & Partial Participation:** Total participation (double line) mandates that every entity occurrence must participate in a relationship (e.g., every MAINTENANCE_LOG must belong to an EQUIPMENT). Partial participation (single line) allows some entities to exist without participating (e.g., a new machine without logs).

- **Strong Entity vs Weak Entity:** A strong entity possesses its own sufficient primary key. A weak entity (MAINTENANCE_LOG) cannot exist without an identifying parent entity (EQUIPMENT) and relies on a partial key (Log_no) combined with the parent's primary key.
- **Composite Primary Identifier:** An identifying key composed of two or more attributes that together uniquely identify a row (e.g., {Class_ID, Visitor_ID} in CLASS_REGISTRATION).

1.1.3 Schedule & Resource Conflicts & Domain Scope

A schedule is a systematic allocation of gym resources and time. The system classifies schedules into two types: Centralized Public Gym Schedules (fixed timetables for group classes published by the gym) and Personal Trainer Schedules (private 1-on-1 coaching slots). A Schedule Conflict occurs when more than one session is scheduled at the same Workout Location during overlapping time intervals [Start_Time, End_Time), OR with the same Personal Trainer during overlapping intervals. Design Note: We selected a two-table model (CLASS and PT_BOOKING) to maintain clean domain separation between public classes and private PT sessions. This requires cross-table dynamic validation enforced via Database Triggers in Phase 2/3.

1.2 Business Rules & Constraints

Note: Codes such as V001, C101, or MEM-501 in the examples are display labels for readability; the stored key values are INT IDENTITY numbers. A member's ID is his/her Visitor_ID (e.g., "Member M001" refers to the same person as Visitor V001).

1.2.1 Member & Membership Domain

BR-MEM-01: Visitor Classification & Total Disjoint Specialization

- **Rule Statement**: Every individual entering the facility is categorized as a VISITOR. A VISITOR must be either a MEMBER or a DAILY_PASS_TICKET holder at any given time (Total & Disjoint constraint: VISITOR = MEMBER $\cup$ DAILY_PASS_TICKET and MEMBER $\cap$ DAILY_PASS_TICKET = $\varnothing$).
- **Technical & EER Mapping**: Modeled as a Total Disjoint Specialization Hierarchy (d) on Superclass VISITOR(Visitor_ID) into Subclasses MEMBER(Visitor_ID) and DAILY_PASS_TICKET(Visitor_ID). Primary Identifier Visitor_ID is inherited by both subclasses as PK/FK. (Relational schema mapping via Option 8A is performed in Phase 2, Section 2.1).

Concrete Example: Visitor "Nguyen Van A" registers for a 1-year package, receiving Visitor_ID = V001 stored in Superclass VISITOR and Subclass MEMBER. Guest "Tran Thi B" buys a single-day pass, receiving Visitor_ID = V002 stored in VISITOR and DAILY_PASS_TICKET. A trigger enforces disjointness: V001 cannot also be inserted into DAILY_PASS_TICKET.

BR-MEM-02: Daily Pass Ticket Access & Feature Restrictions

- **Rule Statement**: DAILY_PASS_TICKET entities represent single-day visitors. Daily pass holders are restricted strictly to general workout area access and cannot register for group classes (CLASS_REGISTRATION) or book personal trainers (PT_BOOKING).

- **Technical & EER Mapping**: Foreign key constraints in CLASS_REGISTRATION and PT_BOOKING reference MEMBER(Visitor_ID), automatically blocking DAILY_PASS_TICKET entities at the database schema level.

Concrete Example: Guest "Tran Thi B" with Visitor_ID = V002 attempts to register for Zumba Class C101. The system rejects the registration because V002 does not exist in table MEMBER.

BR-MEM-03 (Member Account Validation)

- **Rule Statement**: A registered Member must have a profile photo on file before buying a membership or booking a class or PT session.
- **Technical & EER Mapping**: Email is NOT NULL in MEMBER. Photo (VARBINARY(MAX)) may be NULL at registration; a trigger rejects new MEMBERSHIP, CLASS_REGISTRATION, and PT_BOOKING rows for members whose Photo is NULL.

Concrete Example: Member M001 has no photo and tries to book a Yoga class. The booking is rejected until the front desk uploads M001's photo.

BR-MEM-04 (Guest Access Restrictions)

- **Rule Statement**: Guests purchasing a Daily Pass Ticket are permitted facility access strictly for the recorded Visit_Date and cannot purchase long-term memberships or pre-book PT resources.
- **Technical & EER Mapping**: Daily pass holders do not exist in MEMBER, so the FKs of MEMBERSHIP, CLASS_REGISTRATION, and PT_BOOKING (all referencing MEMBER) reject them. Because DAILY_PASS_TICKET uses Visitor_ID as PK, a returning guest keeps the same row and Visit_Date is updated to the new visit; each visit's payment is kept as a separate row in BILLING_LOG. If a guest later becomes a member, the DAILY_PASS_TICKET row is removed and a MEMBER row is created (disjointness).

Concrete Example: Guest V002 (daily pass) tries to book a PT session for tomorrow. The insert into PT_BOOKING fails because V002 does not exist in MEMBER.

BR-MEM-05 (Membership Plans)

- **Rule Statement**: Membership plans (Membership_Plan) define pre-set packages (e.g., Monthly, Yearly, VIP) with specific Price and Duration.
- **Technical & EER Mapping**: Packages are defined once in MEMBERSHIP_PLAN. Attributes: Plan_ID (PK), Plan_Name (UNIQUE), Price (CHECK >= 0.00), Duration_Months (CHECK > 0), Description.

Concrete Example: Plan 'VIP Gold' (Plan_ID = P03) costs $100.00 with Duration_Months = 12. Every membership of P03 refers to this single price record.

BR-MEM-06 (Memberships)

- **Rule Statement**: A membership contract (Membership) links a registered Member to a Membership_Plan over a defined time frame (Start_Date to End_Date).
- **Technical & EER Mapping**: A time-bound relationship entity linking Member and Membership_Plan. Status tracks 'Active', 'Expired', or 'Cancelled'.

Concrete Example: Member M001 buys 'VIP Gold' on 2026-01-01. The system creates Membership_ID = MEM-501 with Start_Date = 2026-01-01, End_Date = 2027-01-01, and Status = 'Active'.

BR-MEM-07: Single Billing Link via Visitor ID

- **Rule Statement**: Every payment in BILLING_LOG is linked to a VISITOR via Visitor_ID [FK, NOT NULL]. This allows one billing table to record payments from both members and daily pass holders.
- **Technical & EER Mapping**: BILLING_LOG contains Visitor_ID FK referencing VISITOR(Visitor_ID), enforced with a NOT NULL constraint, eliminating dual-nullable foreign keys.

Concrete Example: Invoice INV-101 records $15.00 paid by Visitor_ID = V002 (Daily Pass, Membership_ID = NULL). Invoice INV-102 records $100.00 paid by Visitor_ID = V001 (Membership_ID = MEM-501).

BR-MEM-08 (Financial Record Retention)

- **Rule Statement**: Billing records must be retained as historical accounting ledgers and cannot be deleted when a member account or ticket expires.
- **Technical & EER Mapping**: Enforces referential integrity without cascading deletes on financial tables (ON DELETE RESTRICT on Visitor_ID and Membership_ID in BILLING_LOG).

Concrete Example: If visitor V001 is deleted, the RDBMS blocks the DELETE because BILLING_LOG rows reference Visitor_ID = V001.

BR-MEM-09 (Settlement Constraints)

- **Rule Statement**: Each billing record stores Amount_Paid, Billing_Status ('Pending', 'Paid', 'Failed'), Invoice_Date, Due_Date, and Payment_Date.
- **Technical & EER Mapping**: Tracks the invoice lifecycle. Schema constraints: CHECK (Amount_Paid >= 0.00) and CHECK (Billing_Status IN ('Pending', 'Paid', 'Failed')). Payment_Date remains NULL until the invoice is paid.

Concrete Example: Invoice INV-801 is created on 2026-12-28 (Due_Date = 2027-01-01) with Billing_Status = 'Pending' and Payment_Date = NULL. On 2026-12-30 the member pays $100; Billing_Status is updated to 'Paid' and Payment_Date = 2026-12-30.

BR-MEM-10 (Single Active Membership Enforcement)

- **Rule Statement**: At any single timestamp, a registered Member can have at most one active membership contract (Status = 'Active').
- **Technical & EER Mapping**: Enforced via a filtered unique index on MEMBERSHIP(Visitor_ID) WHERE Status = 'Active' (SQL Server filtered index), so the database rejects a second active contract for the same member.

Concrete Example: Member M001 currently has Active Membership MEM-501. If M001 attempts to activate a second membership MEM-502 while MEM-501 is still 'Active', the system rejects MEM-502.

1.2.2 Training & Scheduling Domain

Domain Requirements Analysis:

Guiding Questions: To define the scheduling rules, the team analyzed the following questions:

- **Question for Personal Trainers**: *"Have you ever experienced schedule overlaps between group classes and 1-on-1 personal training sessions?"*
- **Question for Gym Managers**: *"How do room conflicts or workout equipment shortages affect customer satisfaction?"*

Problem Analysis: Based on common gym scheduling scenarios, two conflict types were identified during peak hours (17:00 - 20:00): (1) the same room being booked by a group class and a PT session at the same time, and (2) a trainer being assigned to overlapping sessions. These scenarios shaped rules BR-SCH-01 to BR-SCH-07 below.

BR-SCH-01 (Trainer Specialization)

- **Rule Statement**: Staff members (Staff) are general employees. Using Extended ER (EER) constraints, a Staff member specializing in coaching is modeled as a Trainer subclass.
- **Technical & EER Mapping**: Disjoint partial specialization STAFF (d) -> TRAINER / RECEPTIONIST using Option 8A. Shared attributes (Name, Salary) reside in STAFF; specialized attributes (Specialty, Certification) reside in TRAINER.

Concrete Example: Staff S01 ('Le Van C') has Salary = $800. S01 specializes as Trainer T01 with Specialty = 'Yoga' and Certification = 'NASM-CPT'. A trigger prevents S01 from also being inserted into RECEPTIONIST (disjointness).

BR-SCH-02 (Room Reservation & Capacity)

- **Rule Statement**: A group class (CLASS) or personal training booking (PT_BOOKING) must be scheduled within a designated physical ROOM for a specific time interval. The total number of

confirmed member registrations for a group class cannot exceed the maximum capacity defined for its assigned room (Room.Capacity).

- **Technical & EER Mapping**: Foreign key ROOM.Room_ID in CLASS and PT_BOOKING references ROOM(Room_ID). Attribute ROOM.Capacity carries a CHECK (Capacity > 0) constraint. A BEFORE INSERT/UPDATE trigger on CLASS_REGISTRATION dynamically counts existing 'Confirmed' registrations for CLASS.Class_ID and blocks insertion if COUNT(*) >= ROOM.Capacity.

Concrete Example: Studio A (Room_ID = R101) has Capacity = 20. Zumba Class C101 is scheduled in R101. When member M021 attempts to submit the 21st registration for C101, the trigger blocks the insertion and raises a custom database exception: 'Registration failed: Room capacity reached (20/20)'.

BR-SCH-03 (No Overlapping Room Schedule)

- **Rule Statement**: To prevent physical room double-booking, a single room cannot host more than one event (Class or PT booking) during overlapping time intervals.
- **Technical & EER Mapping**: A cross-table trigger validates proposed interval [Start_Time, End_Time) for Room_ID across CLASS and PT_BOOKING using the overlap condition (S1 < E2 AND S2 < E1). Insertion or update is aborted if an overlap is detected in either table.

Concrete Example: Studio A is booked for Yoga Class C101 from 09:00 to 10:30. A trainer attempting to schedule a private PT Booking B201 in Studio A from 10:00 to 11:00 is blocked.

BR-SCH-04 (No Overlapping Trainer Schedule)

- **Rule Statement**: A Personal Trainer cannot lead a group class and coach a private PT booking during overlapping time intervals, nor accept multiple private bookings simultaneously.
- **Technical & EER Mapping**: Cross-table dynamic validation checks Trainer_ID availability across CLASS and PT_BOOKING for any proposed interval [Start_Time, End_Time) using (S1 < E2 AND S2 < E1). Transgressions abort transaction with 'Trainer Schedule Conflict'.

Concrete Example: Trainer T01 is scheduled to lead Zumba Class C102 from 17:00 to 18:30. Member M002 requests a private PT session with T01 at 18:00. The system rejects the booking due to trainer schedule overlap.

BR-SCH-05 (Class Registration Authorization)

- **Rule Statement**: Class registrations (Class_Registration) are strictly restricted to Members with active memberships (Status = 'Active').
- **Technical & EER Mapping**: Foreign key CLASS_REGISTRATION.Visitor_ID references MEMBER(Visitor_ID). A BEFORE INSERT trigger verifies that the registering member holds a row

in MEMBERSHIP with Status = 'Active' and CURRENT_TIMESTAMP BETWEEN Start_Date AND End_Date.

Concrete Example: Member M001 has active membership MEM-501 and successfully registers for Yoga Class C101. Daily pass holder V002 attempts to register for C101; the FK constraint fails because V002 does not exist in MEMBER. Expired member M099 attempts to register; the trigger rejects insertion because M099 has no row in MEMBERSHIP with Status = 'Active'.

BR-SCH-06 (Session Equipment Reservation & Arc XOR)

- **Rule Statement**: When a Class or PT booking reserves physical equipment, reservations are recorded in SESSION_EQUIPMENT_USAGE under an Exclusive Arc XOR constraint.
- **Technical & EER Mapping**: Associative entity SESSION_EQUIPMENT_USAGE links Equipment_ID to EITHER Class_ID OR Booking_ID, enforcing CHECK ((Class_ID IS NOT NULL AND Booking_ID IS NULL) OR (Class_ID IS NULL AND Booking_ID IS NOT NULL)).

Concrete Example: Pilates Class C105 reserves Pilates Reformer machine E501 (Usage record links to Class_ID = C105, Booking_ID = NULL).

BR-SCH-07 (Equipment Status Dependency)

- **Rule Statement**: If a machine's Operational_Status is 'Broken' or 'Under Maintenance', new reservations of that machine are rejected.
- **Technical & EER Mapping**: A trigger on SESSION_EQUIPMENT_USAGE rejects any INSERT whose Equipment_ID refers to equipment with Operational_Status <> 'Active'.

Concrete Example: Treadmill E101 is set to 'Broken' at 14:00. An attempt to reserve E101 for a 15:00 PT booking is rejected; staff must choose another machine (e.g., E102).

1.2.3 Scheduling Conflict Detection & Boundary Case Analysis

Formal Interval Overlap Condition: Two time intervals [S1, E1) and [S2, E2) overlap if and only if (S1 < E2 AND S2 < E1).

Boundary Case A (Adjacent Non-Overlapping Sessions): Session 1 runs from 09:00:00 to 10:30:00 (E1 = 10:30). Session 2 is requested from 10:30:00 to 11:30:00 (S2 = 10:30). Evaluating the condition: (09:00 < 11:30) IS TRUE, but (10:30 < 10:30) IS FALSE. Since both conditions must be true, Overlap = FALSE. Session 2 is VALID.

Boundary Case B (1-Minute Overlap Conflict): Session 1 runs from 09:00:00 to 10:30:00. Session 2 is requested from 10:29:00 to 11:30:00. Evaluating: (09:00 < 11:30) IS TRUE, and (10:29 < 10:30) IS TRUE. Overlap = TRUE. Session 2 is REJECTED.

1.2.4 Facility & Maintenance Domain

BR-FAC-01 (Asset Identification)

- **Rule Statement**: Fitness equipment (Equipment) is a strong entity identified by Equipment_ID, tracking Equipment_Name, Purchase_Date, and Operational_Status ('Active', 'Under Maintenance', 'Broken').
- **Technical & EER Mapping**: Attributes: Equipment_ID (PK), Equipment_Name, Room_ID (FK, NULL), Purchase_Date, Operational_Status (CHECK Operational_Status IN ('Active', 'Under Maintenance', 'Broken')).

Concrete Example: NordicTrack Treadmill is registered as Equipment_ID = E101, Operational_Status = 'Active', located in Room R102.

BR-FAC-02 (Weak Entity Constraint)

- **Rule Statement**: A maintenance log (Maintenance_Log) cannot exist without a parent equipment and is modeled as a weak entity with partial key Log_no.
- **Technical & EER Mapping**: Maintenance logs are sequential service attempts (1, 2, 3...) dependent on an owner machine. Represented as a double-bordered rectangle in EER.

Concrete Example: Maintenance Log #1 for Treadmill E101 is identified relative to E101. It cannot exist as an independent orphan record in the database. (Log_no has no DEFAULT value to avoid PK collisions across service attempts).

BR-FAC-03 (Composite Primary Identifier)

- **Rule Statement**: The identifying key of Maintenance_Log is a composite key combining owner key and partial key: {Equipment_ID, Log_no}.
- **Technical & EER Mapping**: Primary key of weak entity is composite {Equipment_ID, Log_no}, allowing the same machine to undergo multiple repairs over time without PK duplication.

Concrete Example: Treadmill E101 undergoes repairs in Jan, May, and Aug. The records are identified by {E101, 1}, {E101, 2}, and {E101, 3} respectively. (Log_no is dynamically calculated as MAX(Log_no)+1 without DEFAULT = 1 to prevent Primary Key violations).

BR-FAC-04 (Participation & Dependency)

- **Rule Statement**: The relationship LOGGED_FOR between Equipment and Maintenance_Log has Total Participation (double line) on Maintenance_Log and Partial Participation (single line) on Equipment.

- **Technical & EER Mapping**: Total participation on Maintenance_Log mandates that every log references a valid Equipment_ID. Partial participation on Equipment allows new machines to have 0 logs.

Concrete Example: A newly purchased Cable Crossover machine E205 has zero maintenance logs (valid). Attempting to insert a maintenance log with non-existent Equipment_ID = E999 is blocked.

BR-FAC-05 (Staff Sign-off Chain)

- **Rule Statement**: Every maintenance log must be signed off by exactly one Staff member (specifically a Receptionist or Manager) via Foreign Key Logged_By.
- **Technical & EER Mapping**: Logged_By FK references STAFF(Staff_ID). A trigger checks that the signing staff member has Role IN ('Receptionist', 'Manager').

Concrete Example: Receptionist R01 (Staff_ID = S02) signs off on Maintenance Log {E101, 1} after technician completes belt replacement, recording Logged_By = S02.

BR-FAC-06 (Operational Dependency)

- **Rule Statement**: Equipment whose status is 'Broken' or 'Under Maintenance' cannot be reserved for new sessions.
- **Technical & EER Mapping**: Implemented by the same trigger as BR-SCH-07. Existing future reservations of that machine can be listed with a query for staff to reassign manually.

Concrete Example: Leg Press E301 is set to 'Broken'. New reservations of E301 are rejected, and a query on SESSION_EQUIPMENT_USAGE lists the future bookings that still use E301.

BR-FAC-07 (Financial Validation)

- **Rule Statement**: The maintenance Cost recorded in each log must be a non-negative value (CHECK (Cost >= 0.00)).
- **Technical & EER Mapping**: Validates financial inputs, defaulting Cost = 0.00 if repair is performed under manufacturer warranty.

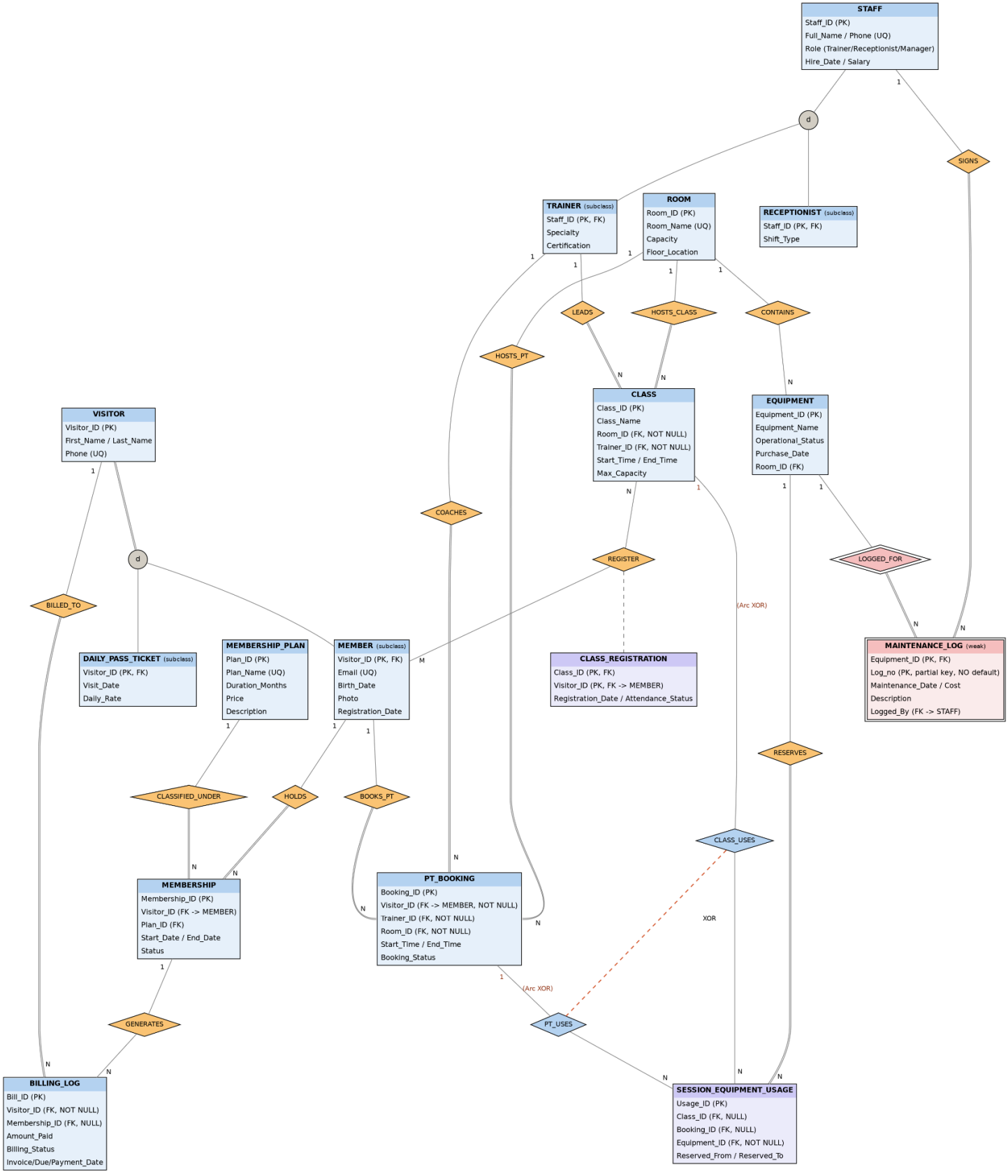
Concrete Example: Routine lubrication on Treadmill E101 under warranty is recorded with Cost = 0.00. Attempting to enter Cost = -50.00 triggers a CHECK constraint violation.

1.3 Conceptual Model (EER Diagram Architectural Notes)

For Phase 1 reporting, our conceptual model is built using Enhanced Entity-Relationship (EER) notation.

Key architectural features include:

- **VISITOR Superclass with Total Disjoint Specialization (d):** Modeled with a double line from VISITOR to the specialization circle, so every visitor is either a MEMBER or a DAILY_PASS_TICKET holder, and both share one Visitor_ID.
- **STAFF Superclass with Disjoint Specialization (d) :** Categorized into **TRAINER** and **RECEPTIONIST** subclasses to manage specialized staff roles and attributes.
- **Billing Table (BILLING_LOG):** Linked to VISITOR via Visitor_ID [FK, NOT NULL] and optionally to MEMBERSHIP via Membership_ID, so one table records payments for both memberships and daily passes.
- **Weak Entity Relationship (EQUIPMENT & MAINTENANCE_LOG):** EQUIPMENT (Strong Entity) is linked to MAINTENANCE_LOG (Weak Entity) via the Identifying Relationship LOGGED_FOR (Double Diamond) with Total Participation on MAINTENANCE_LOG.
- **Exclusive Arc XOR Constraint (SESSION_EQUIPMENT_USAGE):** An associative entity incorporating an Exclusive Arc XOR between Class_ID and Booking_ID to enforce single-purpose equipment reservations.


STAFF
Staff_ID (PK)
Full_Name / Phone (UQ)
Role (Trainer/Receptionist/Manager)
Hire_Date / Salary
d
SIGNS
TRAINER (subclass)
Staff_ID (PK, FK)
Specialty
Certification
ROOM
Room_ID (PK)
Room_Name (UQ)
Capacity
Floor_Location
RECEPTIONIST (subclass)
Staff_ID (PK, FK)
Shift_Type
LEADS
HOSTS_CLASS
CONTAINS
HOSTS_PT
CLASS
Class_ID (PK)
Class_Name
Room_ID (FK, NOT NULL)
Trainer_ID (FK, NOT NULL)
Start_Time / End_Time
Max_Capacity
EQUIPMENT
Equipment_ID (PK)
Equipment_Name
Operational_Status
Purchase_Date
Room_ID (FK)
VISITOR
Visitor_ID (PK)
First_Name / Last_Name
Phone (UQ)
COACHES
REGISTER
LOGGED_FOR
BILLED_TO
d
(Arc XOR)
DAILY_PASS_TICKET (subclass)
Visitor_ID (PK, FK)
Visit_Date
Daily_Rate
MEMBERSHIP_PLAN
Plan_ID (PK)
Plan_Name (UQ)
Duration_Months
Price
Description
MEMBER (subclass)
Visitor_ID (PK, FK)
Email (UQ)
Birth_Date
Photo
Registration_Date
CLASS_REGISTRATION
Class_ID (PK, FK)
Visitor_ID (PK, FK -> MEMBER)
Registration_Date / Attendance_Status
MAINTENANCE_LOG (weak)
Equipment_ID (PK, FK)
Log_no (PK, partial key, NO default)
Maintenance_Date / Cost
Description
Logged_By (FK -> STAFF)
RESERVES
CLASSIFIED_UNDER
HOLDS
BOOKS_PT
CLASS_USES
MEMBERSHIP
Membership_ID (PK)
Visitor_ID (FK -> MEMBER)
Plan_ID (FK)
Start_Date / End_Date
Status
PT_BOOKING
Booking_ID (PK)
Visitor_ID (FK -> MEMBER, NOT NULL)
Trainer_ID (FK, NOT NULL)
Room_ID (FK, NOT NULL)
Start_Time / End_Time
Booking_Status
XOR
(Arc XOR)
GENERATES
PT_USES
BILLING_LOG
Bill_ID (PK)
Visitor_ID (FK, NOT NULL)
Membership_ID (FK, NULL)
Amount_Paid
Billing_Status
Invoice/Due/Payment_Date
SESSION_EQUIPMENT_USAGE
Usage_ID (PK)
Class_ID (FK, NULL)
Booking_ID (FK, NULL)
Equipment_ID (FK, NOT NULL)
Reserved_From / Reserved_To

2. Phase 2: Logical Database Design & Relational Schema Mapping

2.1 Relational Schema Mapping (Elmasri & Navathe's 8-Step Algorithm)

- **Step 1: Mapping Regular Strong Entity Types:** Create a relation R for each strong entity type E. Include all simple attributes. Choose one candidate key as Primary Key (PK):
    - VISITOR(Visitor_ID [PK], First_Name, Last_Name, Phone [UQ])
    - MEMBERSHIP_PLAN(Plan_ID [PK], Plan_Name [UQ], Duration_Months, Price, Description)
    - STAFF(Staff_ID [PK], Full_Name, Phone [UQ], Role, Hire_Date, Salary)
    - ROOM(Room_ID [PK], Room_Name [UQ], Capacity, Floor_Location)
    - EQUIPMENT(Equipment_ID [PK], Equipment_Name, Operational_Status, Purchase_Date)
- **Step 2: Mapping Weak Entity Types:** Create a relation R for each weak entity type W with owner entity type E. Include all simple attributes of W plus the PK of E as a Foreign Key (FK). The PK of R is the composite key combining E's PK and W's partial key:
    - MAINTENANCE_LOG({Equipment_ID [FK -> EQUIPMENT], Log_no} [PK], Maintenance_Date, Cost, Description, Logged_By [FK -> STAFF])
- **Step 3: Mapping Binary 1:1 Relationship Types:** Identify relations S and T. Include PK of S as FK in T. (In our schema, 1:1 relationships are subsumed under 1:N binary relationships or Specialization mapping).
- **Step 4: Mapping Binary 1:N Relationship Types:** Identify relation S (1-side) and T (N-side). Include PK of S as Foreign Key in T:
    - MEMBERSHIP(Membership_ID [PK], Visitor_ID [FK -> MEMBER], Plan_ID [FK -> MEMBERSHIP_PLAN], Start_Date, End_Date, Status)
    - BILLING_LOG(Bill_ID [PK], Visitor_ID [FK -> VISITOR, NOT NULL], Membership_ID [FK -> MEMBERSHIP, NULLABLE], Invoice_Date, Due_Date, Payment_Date, Amount_Paid, Billing_Status)
    - CLASS(Class_ID [PK], Class_Name, Room_ID [FK -> ROOM], Trainer_ID [FK -> TRAINER], Start_Time, End_Time, Max_Capacity)
    - PT_BOOKING(Booking_ID [PK], Visitor_ID [FK -> MEMBER], Trainer_ID [FK -> TRAINER], Room_ID [FK -> ROOM], Start_Time, End_Time, Booking_Status)
    - EQUIPMENT: add Room_ID [FK -> ROOM, NULLABLE] (1:N CONTAINS)

- **Step 5: Mapping Binary M:N Relationship Types:** Create a new associative relation R for M:N relationship type. Include PKs of participating entity types as FKs in R. Their combination forms the composite PK of R:
    - CLASS_REGISTRATION({Class_ID [FK -> CLASS], Visitor_ID [FK -> MEMBER]} [PK], Registration_Date, Attendance_Status)
- **Step 6: Mapping Multivalued Attributes:** Create a new relation for any multivalued attribute. (In our design, multivalued contact attributes like Phone/Email are normalized into UNIQUE atomic columns or managed via primary entity records).
- **Step 7: Mapping N-ary Relationship Types & Associative Entities:** Create relation R for N-ary relationship or associative entity with Exclusive Arc XOR constraints:
    - SESSION_EQUIPMENT_USAGE(Usage_ID [PK], Equipment_ID [FK -> EQUIPMENT], Class_ID [FK -> CLASS, NULLABLE], Booking_ID [FK -> PT_BOOKING, NULLABLE], Reserved_From, Reserved_To)
- **Step 8: Mapping Specialization/Generalization Hierarchies (EER):** Map disjoint specializations using Option 8A (Multiple relations with Same PK/FK):
    - 1) STAFF (d) -> TRAINER / RECEPTIONIST:
        - TRAINER(Staff_ID [PK, FK -> STAFF], Specialty, Certification)
        - RECEPTIONIST(Staff_ID [PK, FK -> STAFF], Shift_Type)
    - 2) VISITOR (d) -> MEMBER / DAILY_PASS_TICKET:
        - MEMBER(Visitor_ID [PK, FK -> VISITOR], Email [UQ], Birth_Date, Photo, Registration_Date)
- DAILY_PASS_TICKET(Visitor_ID [PK, FK -> VISITOR], Visit_Date, Daily_Rate)

2.2 Relational Schema Definition (16 Normalized Tables)

- **VISITOR**: (Visitor_ID [PK], First_Name, Last_Name, Phone [UQ])
    - **MEMBER**: (Visitor_ID [PK/FK->VISITOR], Email [UQ], Birth_Date, Photo, Registration_Date)
    - **DAILY_PASS_TICKET**: (Visitor_ID [PK/FK->VISITOR], Visit_Date, Daily_Rate)
- **MEMBERSHIP_PLAN**: (Plan_ID [PK], Plan_Name [UQ], Duration_Months, Price, Description)
- **MEMBERSHIP**: (Membership_ID [PK], Visitor_ID [FK->MEMBER], Plan_ID [FK->MEMBERSHIP_PLAN], Start_Date, End_Date, Status)
- **BILLING_LOG**: (Bill_ID [PK], Visitor_ID [FK->VISITOR], Membership_ID [FK->MEMBERSHIP, NULL], Invoice_Date, Due_Date, Payment_Date, Amount_Paid, Billing_Status)
- **STAFF**: (Staff_ID [PK], Full_Name, Phone [UQ], Role, Hire_Date, Salary)

    - **TRAINER**: (Staff_ID [PK/FK->STAFF], Specialty, Certification)
    - **RECEPTIONIST**: (Staff_ID [PK/FK->STAFF], Shift_Type)
- **ROOM**: (Room_ID [PK], Room_Name [UQ], Capacity, Floor_Location)
- **CLASS**: (Class_ID [PK], Class_Name, Room_ID [FK->ROOM], Trainer_ID [FK->TRAINER], Start_Time, End_Time, Max_Capacity)
    - **CLASS_REGISTRATION**: (Class_ID [PK/FK->CLASS], Visitor_ID [PK/FK->MEMBER], Registration_Date, Attendance_Status)
    - **PT_BOOKING**: (Booking_ID [PK], Visitor_ID [FK->MEMBER], Trainer_ID [FK->TRAINER], Room_ID [FK->ROOM], Start_Time, End_Time, Booking_Status)
- **EQUIPMENT**: (Equipment_ID [PK], Equipment_Name, Room_ID [FK->ROOM], Operational_Status, Purchase_Date)
    - **MAINTENANCE_LOG**: (Equipment_ID [PK/FK->EQUIPMENT], Log_no [PK], Maintenance_Date, Cost, Description, Logged_By [FK->STAFF])

SESSION_EQUIPMENT_USAGE: (Usage_ID [PK], Equipment_ID [FK->EQUIPMENT], Class_ID [FK->CLASS, NULL], Booking_ID [FK->PT_BOOKING, NULL], Reserved_From, Reserved_To)

2.3 Foreign Key Integrity & Cascading Policy Matrix

Referencing Table (FK)	Referenced Table (PK)	ON DELETE	ON UPDATE	Integrity Rationale
MEMBER (Visitor_ID)	VISITOR (Visitor_ID)	CASCADE	NO ACTION	Subclass record removed if Visitor deleted
DAILY_PASS_TICKET (Visitor_ID)	VISITOR (Visitor_ID)	CASCADE	NO ACTION	Subclass record removed if Visitor deleted
MEMBERSHIP(Visitor_ID)	MEMBER (Visitor_ID)	RESTRICT	NO ACTION	Prevent member deletion with active contract

Referencing Table (FK)	Referenced Table (PK)	ON DELETE	ON UPDATE	Integrity Rationale
MEMBERSHIP(Plan_ID)	MEMBERSHIP_PLAN (Plan_ID)	RESTRICT	NO ACTION	Block deletion of plan in active use
BILLING_LOG (Visitor_ID)	VISITOR (Visitor_ID)	RESTRICT	NO ACTION	Preserve customer financial audit trail
BILLING_LOG (Membership_ID)	MEMBERSHIP(Membership_ID)	RESTRICT	NO ACTION	Preserve MEMBERSHIP billing audit
TRAINER (Staff_ID)	STAFF (Staff_ID)	CASCADE	NO ACTION	Subclass record deleted if Staff removed
RECEPTIONIST (Staff_ID)	STAFF (Staff_ID)	CASCADE	NO ACTION	Subclass record deleted if Staff removed
CLASS (Room_ID)	ROOM (Room_ID)	RESTRICT	NO ACTION	Block room deletion with active classes
CLASS (Trainer_ID)	TRAINER (Staff_ID)	RESTRICT	NO ACTION	Block trainer deletion with scheduled classes
CLASS_REGISTRATION (Class_ID)	CLASS (Class_ID)	CASCADE	NO ACTION	Registrations removed if class cancelled
CLASS_REGISTRATION (Visitor_ID)	MEMBER (Visitor_ID)	CASCADE	NO ACTION	Registrations removed if member deleted
PT_BOOKING (Visitor_ID)	MEMBER (Visitor_ID)	RESTRICT	NO ACTION	Preserve member PT booking history

Referencing Table (FK)	Referenced Table (PK)	ON DELETE	ON UPDATE	Integrity Rationale
PT_BOOKING (Trainer_ID)	TRAINER (Staff_ID)	RESTRICT	NO ACTION	Block trainer deletion with PT bookings
PT_BOOKING (Room_ID)	ROOM (Room_ID)	RESTRICT	NO ACTION	Block room deletion with PT bookings
EQUIPMENT (Room_ID)	ROOM (Room_ID)	SET NULL	NO ACTION	Unassign equipment room location if room deleted, preserving physical asset record
MAINTENANCE_LOG (Equipment_ID)	EQUIPMENT (Equipment_ID)	CASCADE	NO ACTION	Weak entity logs deleted with machine
MAINTENANCE_LOG (Logged_By)	STAFF (Staff_ID)	RESTRICT	NO ACTION	Preserve staff sign-off accountability
SESSION_EQUIPMENT_USAGE (Equipment_ID)	EQUIPMENT (Equipment_ID)	CASCADE	NO ACTION	Reservations deleted if machine removed
SESSION_EQUIPMENT_USAGE (Class_ID)	CLASS (Class_ID)	CASCADE	NO ACTION	Reservations cleared if class cancelled
SESSION_EQUIPMENT_USAGE (Booking_ID)	PT_BOOKING (Booking_ID)	CASCADE	NO ACTION	Reservations cleared if booking cancelled

Implementation note (SQL Server): SQL Server has no RESTRICT keyword; RESTRICT is implemented as NO ACTION, which also blocks the delete. ON UPDATE is NO ACTION for all FKs because every PK is an

IDENTITY surrogate key that is never updated; this also avoids SQL Server error 1785 (multiple cascade paths), e.g., ROOM -> CLASS -> SESSION_EQUIPMENT_USAGE and ROOM -> PT_BOOKING -> SESSION_EQUIPMENT_USAGE.

3. Phase 2: Formal Normalization Analysis (1NF -> 2NF -> 3NF / BCNF)

Formal Definitions:

- **Functional Dependency (FD):** X ⟶ Y holds in R if for any two tuples t1, t2 in R, t1[X] = t2[X] implies t1[Y] = t2[Y].
- **Attribute Closure (X+):** The set of all attributes in R functionally determined by X under FD set F.
- **Superkey Test for BCNF:** A relation R is in BCNF if for every non-trivial FD X ⟶ Y, X is a Superkey of R (i.e., X+ = R).
- **Superkey Test for 3NF:** A relation R is in 3NF if for every non-trivial FD X ⟶ Y, EITHER X is a Superkey of R, OR every attribute A in (Y - X) is a Prime Attribute (member of a Candidate Key).

3.1 Functional Dependency (FD) Sets & Normalization Proofs

• Relation 1: VISITOR

- **Candidate Key(s):** {Visitor_ID}, {Phone}
  FD Set: Visitor_ID -> {First_Name, Last_Name, Phone}; Phone -> Visitor_ID
  Proof & Verdict: {Visitor_ID}+ = VISITOR and {Phone}+ = VISITOR (All attributes). Both left sides are superkeys. BCNF & 3NF Verified.

• Relation 2: MEMBER

- **Candidate Key(s):** {Visitor_ID}, {Email}
  FD Set: Visitor_ID -> {Email, Birth_Date, Photo, Registration_Date}; Email -> Visitor_ID
  Proof & Verdict: {Visitor_ID}+ = MEMBER and {Email}+ = MEMBER (All attributes). Both left sides are superkeys. BCNF & 3NF Verified.

• Relation 3: DAILY_PASS_TICKET

- **Candidate Key(s):** {Visitor_ID}
  FD Set: Visitor_ID -> {Visit_Date, Daily_Rate}
  Proof & Verdict: {Visitor_ID}+ = DAILY_PASS_TICKET (All attributes). Left side Visitor_ID is a Superkey. BCNF & 3NF Verified.

• Relation 4: MEMBERSHIP_PLAN

- **Candidate Key(s):** {Plan_ID}, {Plan_Name}
  FD Set: Plan_ID -> {Plan_Name, Duration_Months, Price, Description}; Plan_Name -> Plan_ID
  Proof & Verdict: {Plan_ID}+ = MEMBERSHIP_PLAN and {Plan_Name}+ = MEMBERSHIP_PLAN (All attributes). Both left sides are superkeys. BCNF & 3NF Verified.

• Relation 5: MEMBERSHIP

- **Candidate Key(s):** {Membership_ID}
  FD Set: Membership_ID -> {Visitor_ID, Plan_ID, Start_Date, End_Date, Status}
  Proof & Verdict: {Membership_ID}+ = MEMBERSHIP (All attributes). Left side Membership_ID is a Superkey. BCNF & 3NF Verified.

• Relation 6: BILLING_LOG

- **Candidate Key(s):** {Bill_ID}
  FD Set: Bill_ID -> {Visitor_ID, Membership_ID, Invoice_Date, Due_Date, Payment_Date, Amount_Paid, Billing_Status}
  Proof & Verdict: {Bill_ID}+ = BILLING_LOG (All attributes). Left side Bill_ID is a Superkey. BCNF & 3NF Verified.
- Note: For rows where Membership_ID is not NULL, Membership_ID -> Visitor_ID also holds (via MEMBERSHIP), so Visitor_ID is partly redundant and, strictly, BILLING_LOG is not fully 3NF for those rows. This is a deliberate design choice: Visitor_ID must be NOT NULL so that daily-pass payments (Membership_ID = NULL) are still linked to a payer. The redundancy is controlled by a trigger requiring BILLING_LOG.Visitor_ID = MEMBERSHIP.Visitor_ID whenever Membership_ID is not NULL.

• Relation 7: STAFF

- **Candidate Key(s):** {Staff_ID}, {Phone}
  FD Set: Staff_ID -> {Full_Name, Phone, Role, Hire_Date, Salary}; Phone -> Staff_ID
  Proof & Verdict: {Staff_ID}+ = STAFF and {Phone}+ = STAFF (All attributes). Both left sides are superkeys. BCNF & 3NF Verified.

• Relation 8: TRAINER

- **Candidate Key(s):** {Staff_ID}
  FD Set: Staff_ID -> {Specialty, Certification}
  Proof & Verdict: {Staff_ID}+ = TRAINER (All attributes). Left side Staff_ID is a Superkey. BCNF & 3NF Verified.

• Relation 9: RECEPTIONIST

- **Candidate Key(s):** {Staff_ID}
  FD Set: Staff_ID -> {Shift_Type}
  Proof & Verdict: {Staff_ID}+ = RECEPTIONIST (All attributes). Left side Staff_ID is a Superkey. BCNF & 3NF Verified.

• Relation 10: ROOM

- **Candidate Key(s):** {Room_ID}, {Room_Name}
  FD Set: Room_ID -> {Room_Name, Capacity, Floor_Location}; Room_Name -> Room_ID
  Proof & Verdict: {Room_ID}+ = ROOM and {Room_Name}+ = ROOM (All attributes). Both left sides are superkeys. BCNF & 3NF Verified.

• Relation 11: CLASS

- **Candidate Key(s):** {Class_ID}
  FD Set: Class_ID -> {Class_Name, Room_ID, Trainer_ID, Start_Time, End_Time, Max_Capacity}

Proof & Verdict: {Class_ID}+ = CLASS (All attributes). Left side Class_ID is a Superkey. BCNF & 3NF Verified.

• Relation 12: CLASS_REGISTRATION

- **Candidate Key(s):** {Class_ID, Visitor_ID}
  FD Set: {Class_ID, Visitor_ID} -> {Registration_Date, Attendance_Status}
  Proof & Verdict: {Class_ID, Visitor_ID}+ = CLASS_REGISTRATION (All attributes). The composite key is a superkey. BCNF & 3NF Verified.

• Relation 13: PT_BOOKING

- **Candidate Key(s):** {Booking_ID}
  FD Set: Booking_ID -> {Visitor_ID, Trainer_ID, Room_ID, Start_Time, End_Time, Booking_Status}
  Proof & Verdict: {Booking_ID}+ = PT_BOOKING (All attributes). Left side Booking_ID is a Superkey. BCNF & 3NF Verified.

• Relation 14: EQUIPMENT

- **Candidate Key(s):** {Equipment_ID}
  FD Set: Equipment_ID -> {Equipment_Name, Room_ID, Operational_Status, Purchase_Date}
  Proof & Verdict: {Equipment_ID}+ = EQUIPMENT (All attributes). Left side Equipment_ID is a Superkey. BCNF & 3NF Verified.

• Relation 15: MAINTENANCE_LOG

- **Candidate Key(s):** {Equipment_ID, Log_no}
  FD Set: {Equipment_ID, Log_no} -> {Maintenance_Date, Cost, Description, Logged_By}
  Proof & Verdict: {Equipment_ID, Log_no}+ = MAINTENANCE_LOG (All attributes). Left side Composite Key is a Superkey. BCNF & 3NF Verified.

• Relation 16: SESSION_EQUIPMENT_USAGE

- **Candidate Key(s):** {Usage_ID}
- **FD Set:** Usage_ID -> {Equipment_ID, Class_ID, Booking_ID, Reserved_From, Reserved_To}
- **Proof & Verdict:** {Usage_ID}+ = SESSION_EQUIPMENT_USAGE (All attributes). Left side Usage_ID is a Superkey. BCNF & 3NF Verified.

3.2 Step-by-Step Normalization Proofs (1NF -> 2NF -> 3NF / BCNF)

- **1NF Compliance Proof:** All attributes in all 16 relations contain atomic, single-valued primitive types (INT, VARCHAR, VARBINARY, DATETIME, DECIMAL, DATE). No multi-valued attributes or repeating groups exist.
- **2NF Compliance Proof:** Every non-prime attribute in every table is fully functionally dependent on the primary key. In MAINTENANCE_LOG with composite key {Equipment_ID, Log_no}, attributes Cost and Maintenance_Date depend on the full composite key, not a proper subset.

3NF & BCNF Compliance Proof: For every non-trivial functional dependency X ⟶ Y in the 16 tables, X is a superkey, with one documented exception: BILLING_LOG keeps a controlled redundancy

(Membership_ID -> Visitor_ID, see Relation 6) to support daily-pass payments. All other relations are in BCNF.

4. Data Dictionary & Metadata Specifications

Table 1: VISITOR (Superclass Entity)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Visitor_ID	INT	PK	NO	IDENTITY	Primary identifier for all visitors
First_Name	VARCHAR(50)	None	NO	None	Visitor first name
Last_Name	VARCHAR(50)	None	NO	None	Visitor last name
Phone	VARCHAR(15)	UNIQUE	NO	None	Contact phone number

Table 2: MEMBER (Registered Members Subclass)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
Visitor_ID	INT	PK, FK -> VISITOR	NO	None	Subclass PK referencing VISITOR(Visitor_ID)
Email	VARCHAR(100)	UNIQUE	NO	None	Member electronic mail address

Birth_Date	DATE	None	YES	NULL	Date of birth for age verification
Photo	VARBINARY(MAX)	None	YES	NULL	Member profile photo stored as binary data (NULL until uploaded)
Registration_Date	DATETIME	None	NO	CURRENT_TIMESTAMP	Member contract registration timestamp

Table 3: DAILY_PASS_TICKET (Walk-in Pass Subclass)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Visitor_ID	INT	PK, FK -> VISITOR	NO	None	Subclass PK referencing VISITOR(Visitor_ID)
Visit_Date	DATE	None	NO	CAST(GETDATE() AS DATE)	Single-visit gym access date
Daily_Rate	DECIMAL(10,2)	None	NO	15.00	Ticket entrance fee (CHECK Daily_Rate > 0.00)

Table 4: MEMBERSHIP_PLAN (Standard Plan Catalog)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints

Plan_ID	INT	PK	NO	IDENTITY	Unique identifier for membership plan
Plan_Name	VARCHAR(50)	UNIQUE	NO	None	Plan title (Monthly, Annual, VIP)
Duration_Months	INT	None	NO	1	Active duration (CHECK Duration_Months > 0)
Price	DECIMAL(10,2)	None	NO	0.00	Package price (CHECK Price >= 0.00)
Description	VARCHAR(255)	None	YES	NULL	Summary of plan benefits

Table 5: MEMBERSHIP (Member Contracts)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
Membership_ID	INT	PK	NO	IDENTITY	Unique contract identifier
Visitor_ID	INT	FK -> MEMBER	NO	None	Refers to registered Member(Visitor_ID)
Plan_ID	INT	FK -> MEMBERSHIP_PLAN	NO	None	Refers to selected Membership_Plan
Start_Date	DATE	None	NO	None	Contract start date

End_Date	DATE	None	NO	None	Contract end date (CHECK End_Date > Start_Date)
Status	VARCHAR(20)	None	NO	'Active'	State: CHECK (Status IN ('Active','Expired','Cancelled'))

Table 6: BILLING_LOG (Billing Records)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
Bill_ID	INT	PK	NO	IDENTITY	Unique invoice transaction ID
Visitor_ID	INT	FK -> VISITOR	NO	None	Refers to paying VISITOR(Visitor_ID)
Membership_ID	INT	FK -> MEMBERSHIP	YES	NULL	Refers to Membership (NULL for Daily Pass). When not NULL, must belong to the same Visitor_ID (checked by trigger)
Invoice_Date	DATETIME	None	NO	CURRENT_TIMESTAMP	Invoice issuance date
Due_Date	DATE	None	NO	None	Payment due date
Payment_Date	DATETIME	None	YES	NULL	Actual payment timestamp

Amount_Paid	DECIMAL(10, 2)	None	NO	0.00	Amount paid (CHECK Amount_Paid >= 0.00)
Billing_Status	VARCHAR(20)	None	NO	'Pending'	Status: CHECK (Billing_Status IN ('Pending','Paid','Failed'))

Table 7: STAFF (Employee Superclass)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
Staff_ID	INT	PK	NO	IDENTITY	Primary identifier for gym employee
Full_Name	VARCHAR(100)	None	NO	None	Employee full name
Phone	VARCHAR(15)	UNIQUE	NO	None	Employee phone number
Role	VARCHAR(30)	None	NO	'Receptionist'	Role: CHECK (Role IN ('Trainer','Receptionist','Manager'))
Hire_Date	DATE	None	NO	CAST(GETDATE() AS DATE)	Date of employment
Salary	DECIMAL(10,2)	None	NO	500.00	Monthly salary (CHECK Salary > 0.00)

Table 8: TRAINER (Personal Trainers Subclass)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints

Staff_ID	INT	PK, FK -> STAFF	NO	None	Subclass PK referencing STAFF(Staff_ID)
Specialty	VARCHAR(50)	None	NO	'General'	Coaching field (Yoga, Strength, HIIT)
Certification	VARCHAR(100)	None	YES	NULL	Title of fitness qualification

Table 9: RECEPTIONIST (Front-Desk Staff Subclass)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
Staff_ID	INT	PK, FK -> STAFF	NO	None	Subclass PK referencing STAFF(Staff_ID)
Shift_Type	VARCHAR(20)	None	NO	'Morning'	Shift: CHECK (Shift_Type IN ('Morning','Evening','Night'))

Table 10: ROOM (Gym Facilities & Locations)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Room_ID	INT	PK	NO	IDENTITY	Unique identifier for physical room/area
Room_Name	VARCHAR(50)	UNIQUE	NO	None	Room title (Studio A, Pilates Room)

Capacity	INT	None	NO	20	Max physical capacity (CHECK Capacity > 0)
Floor_Location	VARCHAR(30)	None	YES	'Floor 1'	Floor location in facility

Table 11: CLASS (Public Group Classes)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Class_ID	INT	PK	NO	IDENTITY	Unique identifier for group class
Class_Name	VARCHAR(50)	None	NO	None	Title of class (Zumba, Yoga, HIIT)
Room_ID	INT	FK -> ROOM	NO	None	Refers to physical room location
Trainer_ID	INT	FK -> TRAINER	NO	None	Refers to assigned instructor
Start_Time	DATETIME	None	NO	None	Scheduled start timestamp
End_Time	DATETIME	None	NO	None	Scheduled end timestamp (CHECK End_Time > Start_Time)
Max_Capacity	INT	None	NO	20	Max allowed attendees (CHECK

					Max_Capacity > 0; must not exceed ROOM.Capacity, checked by trigger)

Table 12: CLASS_REGISTRATION (Member Group Enrollments)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Registration_ID	INT	PK	NO	IDENTITY	Unique registration record ID
Class_ID	INT	FK -> CLASS	NO	None	Refers to registered group class
Visitor_ID	INT	FK -> MEMBER	NO	None	Refers to registering Member(Visitor_ID)
Registration_Date	DATETIME	None	NO	CURRENT_TIMESTAMP	Timestamp when registration occurred
Attendance_Status	VARCHAR(20)	None	NO	'Confirmed'	Status: CHECK (Attendance_Status IN ('Confirmed','Attended','Cancelled'))

Table 13: PT_BOOKING (1-on-1 Coaching Sessions)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Booking_ID	INT	PK	NO	IDENTITY	Unique identifier for private PT session

Visitor_ID	INT	FK -> MEMBER	NO	None	Refers to registered Member(Visitor_ID)
Trainer_ID	INT	FK -> TRAINER	NO	None	Refers to assigned Personal Trainer
Room_ID	INT	FK -> ROOM	NO	None	Refers to reserved workout space
Start_Time	DATETIME	None	NO	None	Scheduled start timestamp
End_Time	DATETIME	None	NO	None	Scheduled end timestamp (CHECK End_Time > Start_Time)
Booking_Status	VARCHAR(20)	None	NO	'Scheduled'	Status: CHECK (Booking_Status IN ('Scheduled','Completed','Cancelled'))

Table 14: EQUIPMENT (Workout Hardware Assets)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Equipment_ID	INT	PK	NO	IDENTITY	Unique hardware machine ID
Equipment_Name	VARCHAR(100)	None	NO	None	Machine brand/model name
Room_ID	INT	FK -> ROOM	YES	None	Refers to physical room location
Operational_Status	VARCHAR(20)	None	NO	'Active'	Status: CHECK (Operational_Status IN ('Active','Under

					Maintenance','Broken'))
Purchase_Date	DATE	None	YES	NULL	Acquisition date of equipment

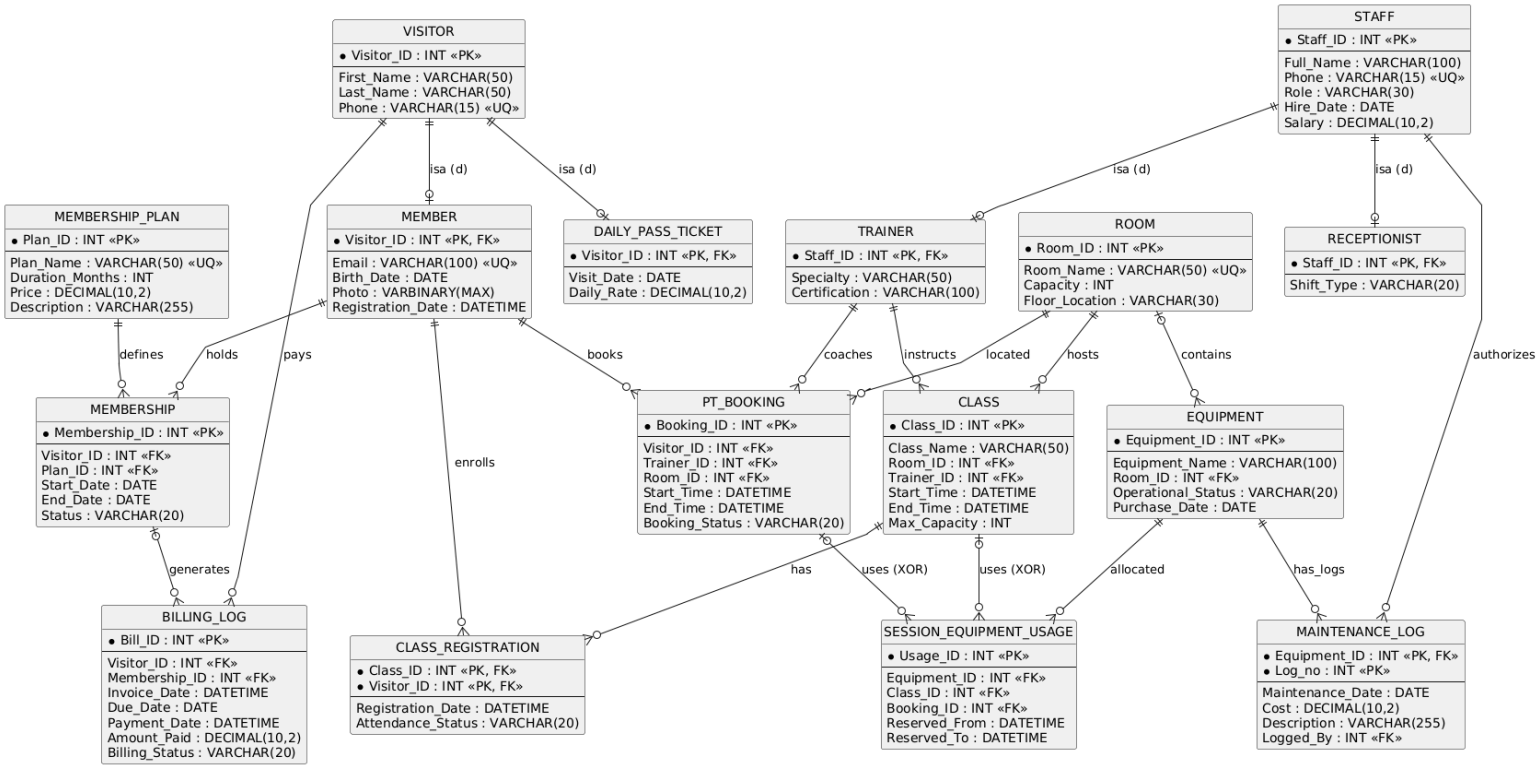
Table 15: MAINTENANCE_LOG (Weak Entity Service History)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Equipment_ID	INT	PK, FK -> EQUIPMENT	NO	None	Composite PK part 1 referencing parent machine
Log_no	INT	PK (Partial Key)	NO	None	Composite PK part 2 (Sequential log number per asset)
Maintenance_Date	DATE	None	NO	CAST(GETDATE() AS DATE)	Date service was performed
Cost	DECIMAL(10,2)	None	NO	0.00	Repair cost (CHECK Cost >= 0.00)
Description	VARCHAR(255)	None	NO	None	Summary of service performed
Logged_By	INT	FK -> STAFF	NO	None	Refers to Staff/Receptionist signing off

Table 16: SESSION_EQUIPMENT_USAGE (Equipment Allocation & Arc XOR)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
Usage_ID	INT	PK	NO	IDENTITY	Unique usage reservation ID
Equipment_ID	INT	FK -> EQUIPMENT	NO	None	Refers to allocated fitness machine
Class_ID	INT	FK -> CLASS	YES	NULL	Refers to Class (Arc XOR with Booking_ID)
Booking_ID	INT	FK -> PT_BOOKING	YES	NULL	Refers to PT_Booking (Arc XOR with Class_ID)

2.4 Physical Database Schema (IE Crow's Foot Notation)



5. Conclusion & Recommendations

📌 **GHI CHÚ MỤC ĐÍCH / PURPOSE NOTE:** *Tổng kết kiến trúc CSDL, khẳng định tính toàn vẹn dữ liệu và đề xuất định hướng kỹ thuật cho Phase 3 (Cài đặt CSDL vật lý bằng SQL DDL Script, Indexes và Triggers).*